Assignment 2: iPerf, TCP Statistics, and Congestion Control

Annsh Navle, Rohan Malpani, Valmik Vyas
CS 422 – Fall 2026

Experiment repository:
https://github.com/RohMalpani/NetworksAssignment1/tree/main/assignment2

1 Part 1: iPerf Throughput Application

1.1 (a) Client implementation

Our Python socket client performs a single-stream TCP upload to standard public iperf3 servers without invoking the iperf3 binary or using a custom receiver. `run_test` opens a control connection, exchanges JSON test parameters, and opens a data connection using the same session cookie. After the server signals that the test is running, `_send_data` continuously sends data for the configured duration. The client sends `TEST_END`, exchanges results, and finishes with `IPERF_DONE`. Connection and I/O timeouts are five and ten seconds, respectively. Rejections, unexpected states, closed connections, and socket errors are logged as failures.

1.2 (b) Server selection and setup

`ServerPool` randomly shuffles entries from `servers.csv`, sourced from iperf3serverlist.net, and selects entries without replacement, choosing a random port from each advertised range. The driver replaces failed destinations until the requested count is reached, the list is exhausted, or ten attempts have failed.

Run `20261017-181935` contains ten successful destinations, one 60-second transfer per destination, and a nominal one-second sampling interval. All successful data sockets reported CUBIC via `TCP_CONGESTION`. There were two failed attempts: Vienna (`185.180.12.40:5201`) returned `ACCESS_DENIED`, and Pune (`69.48.236.198:30004`) returned a broken-pipe error. Both were replaced; there was no destination shortfall. Selected addresses, ports, and attempt outcomes are recorded in `part1_runs.csv`. The project log records WSL2, Ubuntu 22.04, Python 3.10, and kernel series 6.6; the exact kernel release was not saved.

1.3 (c) Goodput measurement

`GoodputSampler` reads cumulative `tcpi_bytes_acked` through `getsockopt(TCP_INFO)` on the *data socket*. For acknowledged bytes A_i and actual elapsed time t_i,

$$G_i = \frac{8(A_i - A_{i-1})}{t_i - t_{i-1}} \text{ bits/s.}$$

Using acknowledged bytes avoids counting data only buffered by `send()`. Actual sampling intervals, rather than the nominal one second, are used. The initial counter is zero, so the first interval can include a small protocol byte offset. Raw samples are saved in `part1_samples.csv`.

Results and observations

Destination	Algorithm	Min	Median	Mean	P95	N
Algiers (41.110.39.130)	cubic	2.53	177.12	154.12	179.28	60
Alvsjo (iperf.fegis.nu)	cubic	1.46	136.51	108.54	140.50	60
Helsinki (spd-fisrv.hostkey.com)	cubic	1.58	158.46	134.46	181.32	60
Istanbul (69.48.237.66)	cubic	1.33	153.80	123.70	155.75	60
London (96.45.40.45)	cubic	7.09	20.86	26.83	62.53	60
Montreal (speedtest.mtl2.ca.leaseweb.net)	cubic	303.85	453.87	451.07	461.82	60
Paris (ping-90ms.online.net)	cubic	1.06	69.45	61.85	110.17	60
Phoenix (phx.speedtest.clouvider.net)	cubic	225.01	378.25	376.04	392.22	60
Plano (209.40.123.215)	cubic	293.76	436.40	440.43	472.52	60
Reykjavik (spd-icsrv.hostkey.com)	cubic	12.85	167.07	164.46	170.12	60

Table 1: Goodput in Mbps for CUBIC. Each destination has 60 interval samples. Statistics include startup; the mean weights samples equally, and P95 uses linear interpolation. Generated by `summary_table`.

Montreal achieved the highest mean goodput (451.07 Mbps), followed by Plano (440.43 Mbps) and Phoenix (376.04 Mbps). London had the lowest mean (26.83 Mbps). These results describe the observed paths during the experiment, not the servers' maximum capacities.

Algiers and Istanbul ramped up before stabilizing near 179 and 154 Mbps, respectively. Paris reached approximately 108 Mbps only after about 34 seconds, which reduced its full-run mean to 61.85 Mbps. This startup behavior is consistent with TCP increasing its sending window. London showed an early spike followed by a substantial decline, while Helsinki had intermittent drops. Goodput alone cannot distinguish congestion, loss recovery, and receiver limits. With only one run per destination, these results do not establish repeatability.

AI acknowledgment. Claude Code assisted with implementation and debugging.

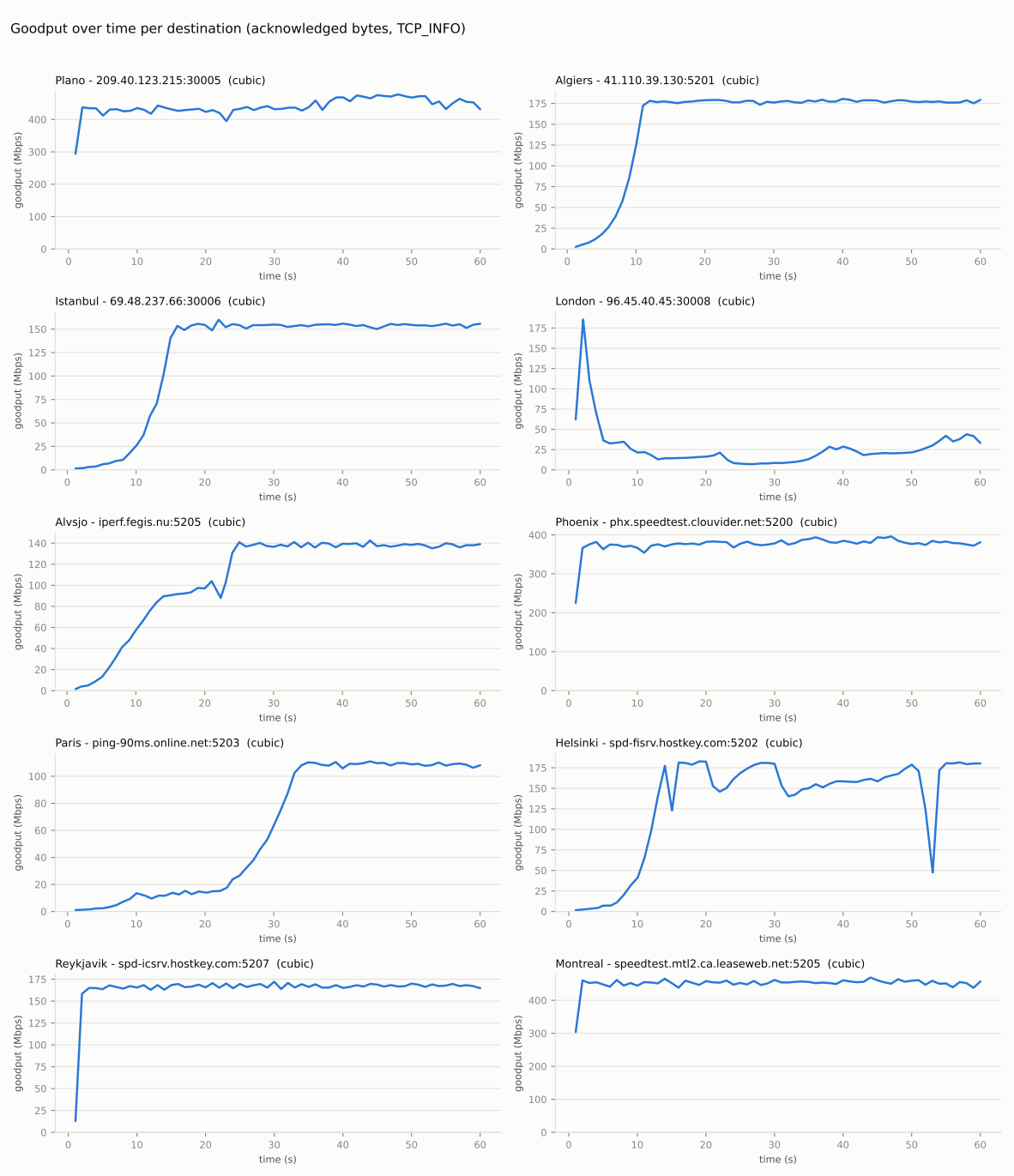


Figure 1: Goodput over time for all ten destinations, generated by `plot_goodput`. Each panel uses its own vertical scale and labels the server address, port, and actual algorithm.

Automation and reproducibility

run_all.py automatically selects servers, collects measurements, and generates PDF plots and summary tables. Run on Linux or WSL2. Use servers.csv, with IP/HOST and PORT columns (single ports or ranges). requirements.txt lists Matplotlib ≥ 3.7, pandas ≥ 2.0, and NumPy ≥ 1.24.

Option	Meaning
--servers	Server-list CSV
-n	Number of successful destinations
--duration	Transfer duration per test (seconds)
--interval	Sampling interval (seconds)
--reps	Repetitions per destination
--algorithm	Requested congestion-control algorithm
--out-dir, --plots-dir	Data/table and PDF output directories

From the repository root, install dependencies and run the Part 1 configuration:

```
cd assignment2
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
python3 run_all.py --servers servers.csv -n 10 \
  --duration 60 --interval 1 --reps 1 --algorithm cubic \
  --out-dir results/new --plots-dir plots/new
```

For all three algorithms, use the same entry point with separate output folders. CUBIC, Reno, and BBR must be available in the host Linux kernel.

```
for cc in cubic reno bbr; do
  python3 run_all.py --servers servers.csv -n 10 \
    --duration 60 --interval 1 --reps 3 --seed 0 \
    --algorithm "$cc" --out-dir "results/$cc" \
    --plots-dir "plots/$cc"
done
```

The saved report results are CUBIC-only. Here, --seed 0 fixes candidate selection; server failures can still change the final destinations across algorithms.

Outputs are part1_runs.csv (attempts), part1_samples.csv (raw measurements), part1_summary.csv and part1_summary.tex (statistics), and part1_goodput.pdf (plots). To regenerate the submitted outputs from the saved data and compile the report:

```
python3 run_all.py --replot-only
```

2 Part 2: TCP Statistics During Transfer

2.1 (a) Measurements and decoding

Part 2 reuses the live data-socket measurements collected in Part 1. The representative destination is Algiers (`41.110.39.130:5201`), selected by the plotting script as the destination whose mean goodput is closest to the median of the destination means. This trace contains 60 samples from one 60-second run; the recorded algorithm is CUBIC.

`read_tcp_info` requests 256 bytes using `getsockopt(TCP_INFO)`. An ordered field table mirrors Linux's `struct tcp_info`, using native byte order and fixed-width unsigned 8-, 32-, and 64-bit fields; decoding stops at the returned buffer length. The project log records validation against C field offsets on WSL2.

Each CSV sample records a timestamp, elapsed interval, congestion window (`snd_cwnd`, segments), smoothed RTT (`rtt_us`, microseconds), and goodput (bits/s). It also includes RTT variation, acknowledged bytes, pacing and delivery rates, destination, run identifier, repetition, and actual algorithm. The loss proxy is

$$L_i = \text{total_retrans}_i - \text{total_retrans}_{i-1}.$$

This counts retransmitted segments during an interval, not a packet-loss percentage; retransmissions can also be spurious. Goodput uses the actual elapsed interval as described in Part 1.

2.2 (b) Visualization

`plot_tcp_stats` produces four time series and three scatter plots from the same samples. The following figures show window, RTT, retransmissions, and goodput over time, followed by window, RTT, and retransmissions against goodput. RTT is displayed in ms and goodput in Mbps. The single entry point now generates these plots as well:

```
python3 run_all.py --replot-only
```

The output is `plots/part2_tcp_stats.pdf`. Select another destination using `--representative` followed by `HOST:PORT`.

2.3 (c) Interpretation and observations

The congestion window limits unacknowledged data. When window-limited, the sending rate is approximately window bytes divided by RTT; increasing the window can therefore raise goodput, while higher RTT can lower it. Retransmission recovery can reduce the window and useful throughput. Other limits, such as path capacity or the receiver window, can prevent further gains.

In this trace, the window grows from 70 to 2,361 segments by 11 seconds, while goodput rises from 2.53 to 172.41 Mbps. The window then stays constant and goodput stabilizes near 177–179 Mbps. The window–goodput scatter reflects this startup growth and plateau; it does not isolate congestion avoidance. RTT initially measures 195.15 ms, then drops to about 122 ms by the third sample and remains broadly stable. This early change is an observed transient; the trace does not establish its cause.

All retransmission intervals are zero, so the loss scatter collapses to a vertical line at zero. This run cannot establish how loss affects goodput. Likewise, the RTT scatter mixes startup and steady operation, so its association with goodput should not be interpreted as causal. One run cannot establish repeatability.

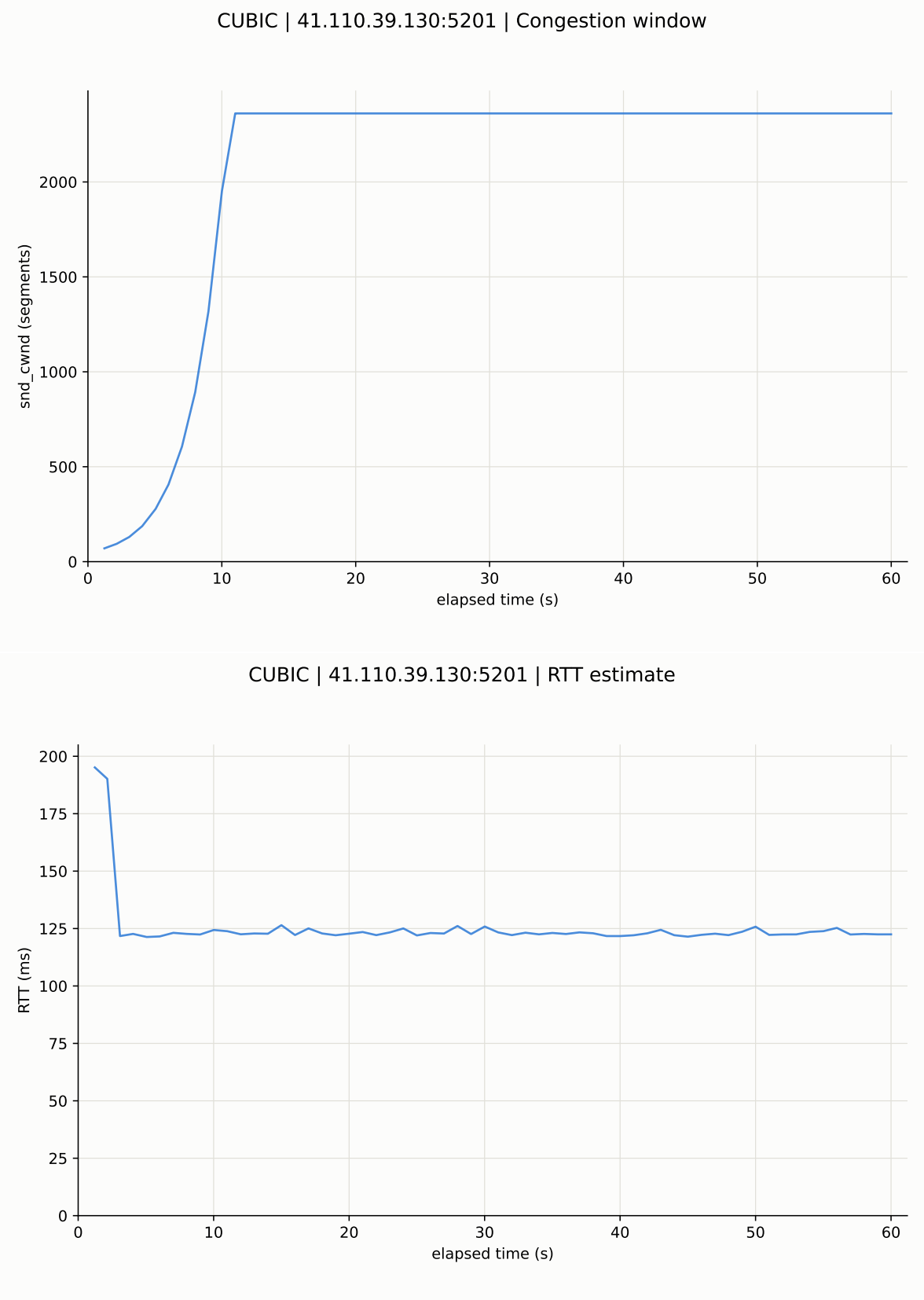
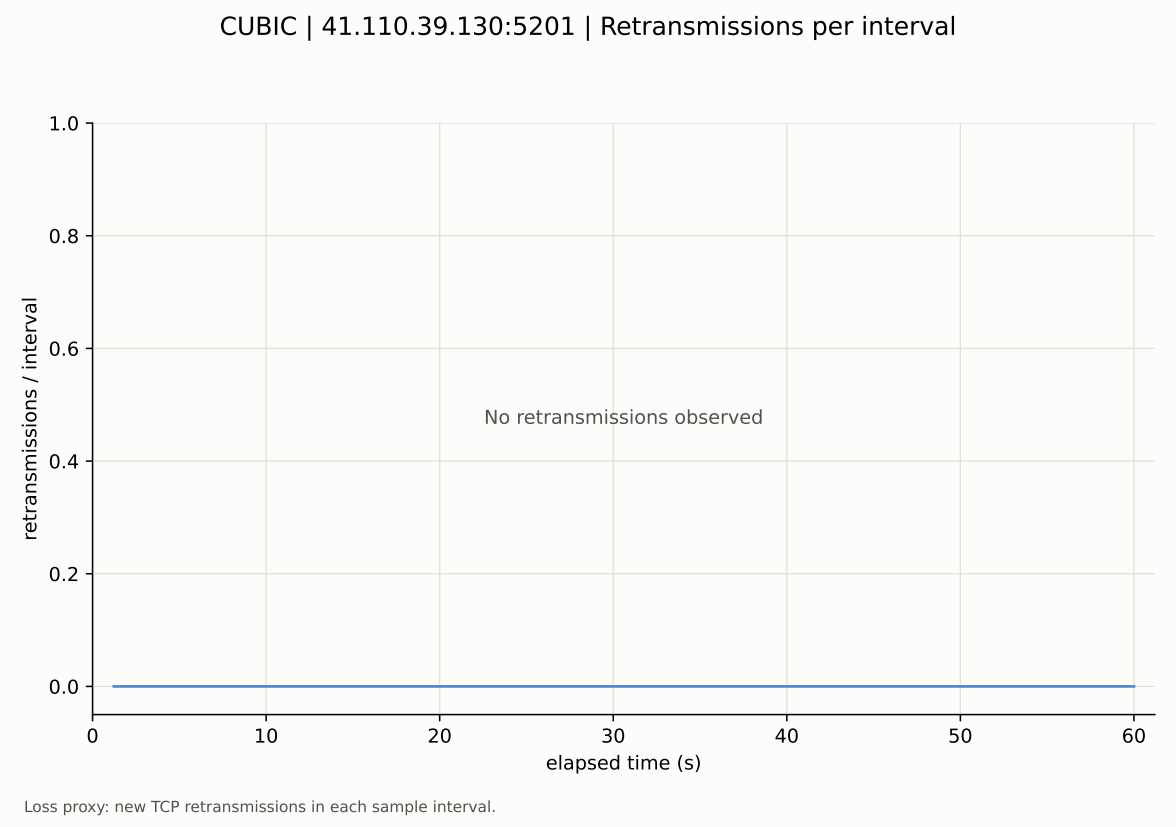


Figure 2: Algiers, CUBIC: congestion window and RTT over time.



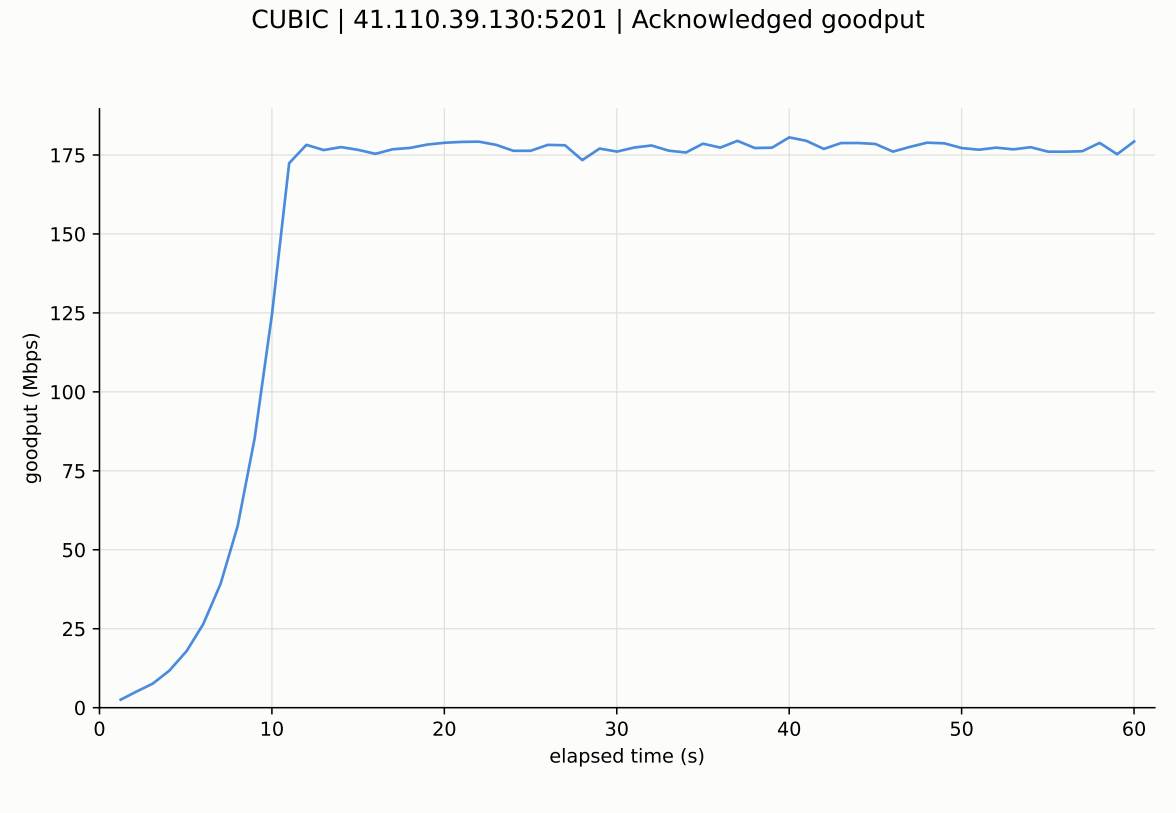


Figure 3: Algiers, CUBIC: interval retransmissions and acknowledged goodput.

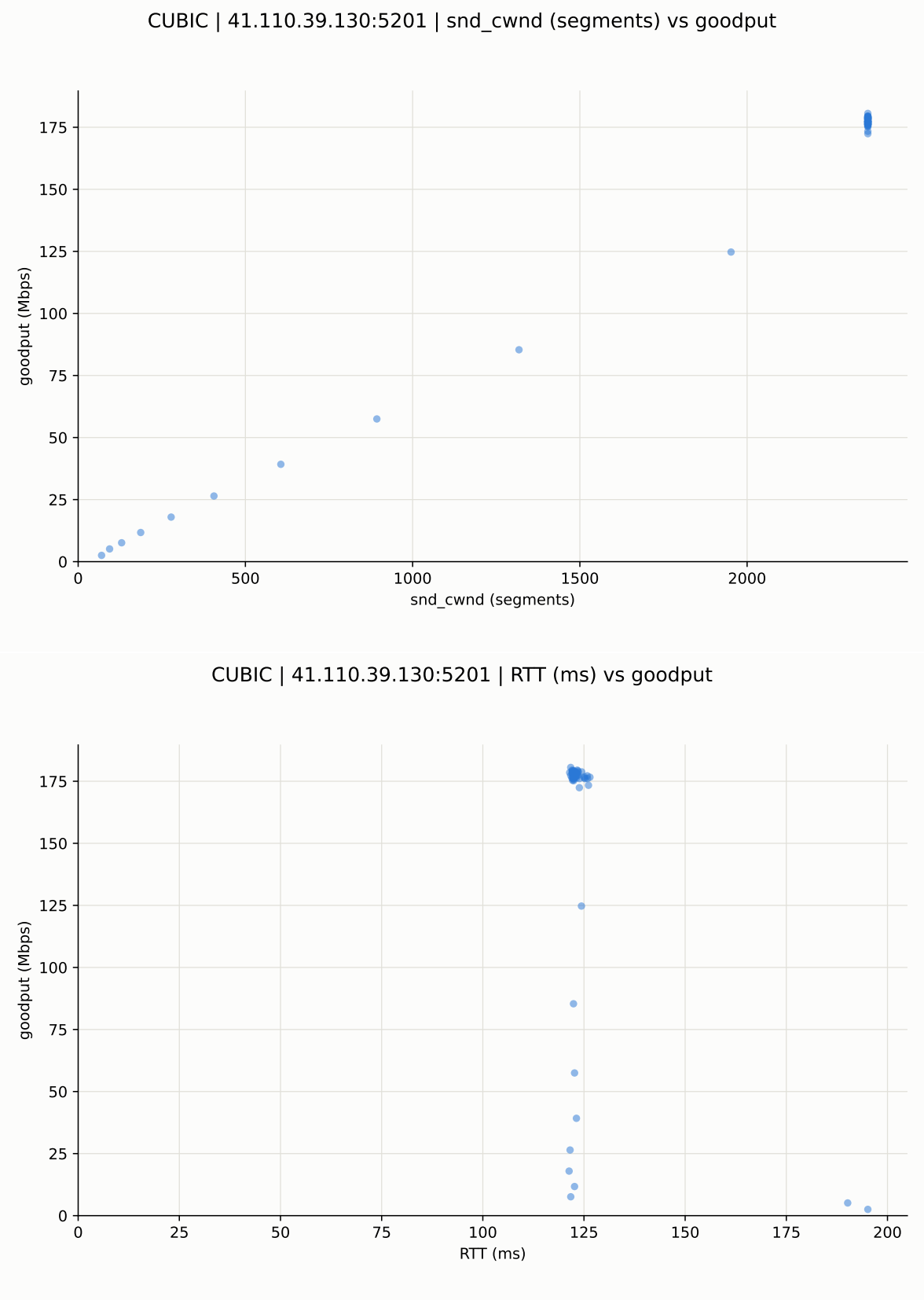


Figure 4: Algiers, CUBIC: congestion window and RTT versus goodput.

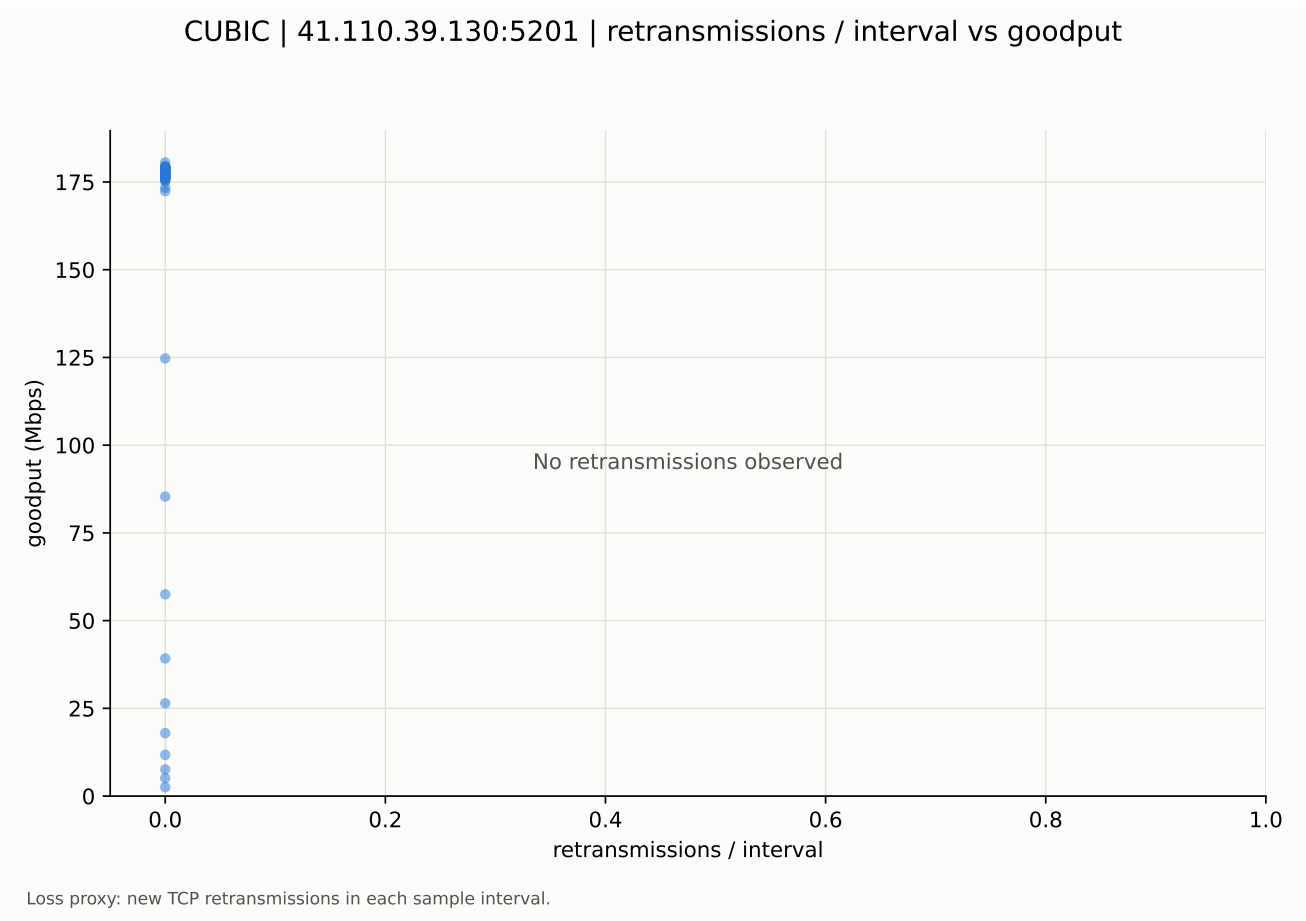


Figure 5: Algiers, CUBIC: retransmissions versus goodput. No retransmissions were recorded, so the samples cannot reveal a loss–goodput relationship.